



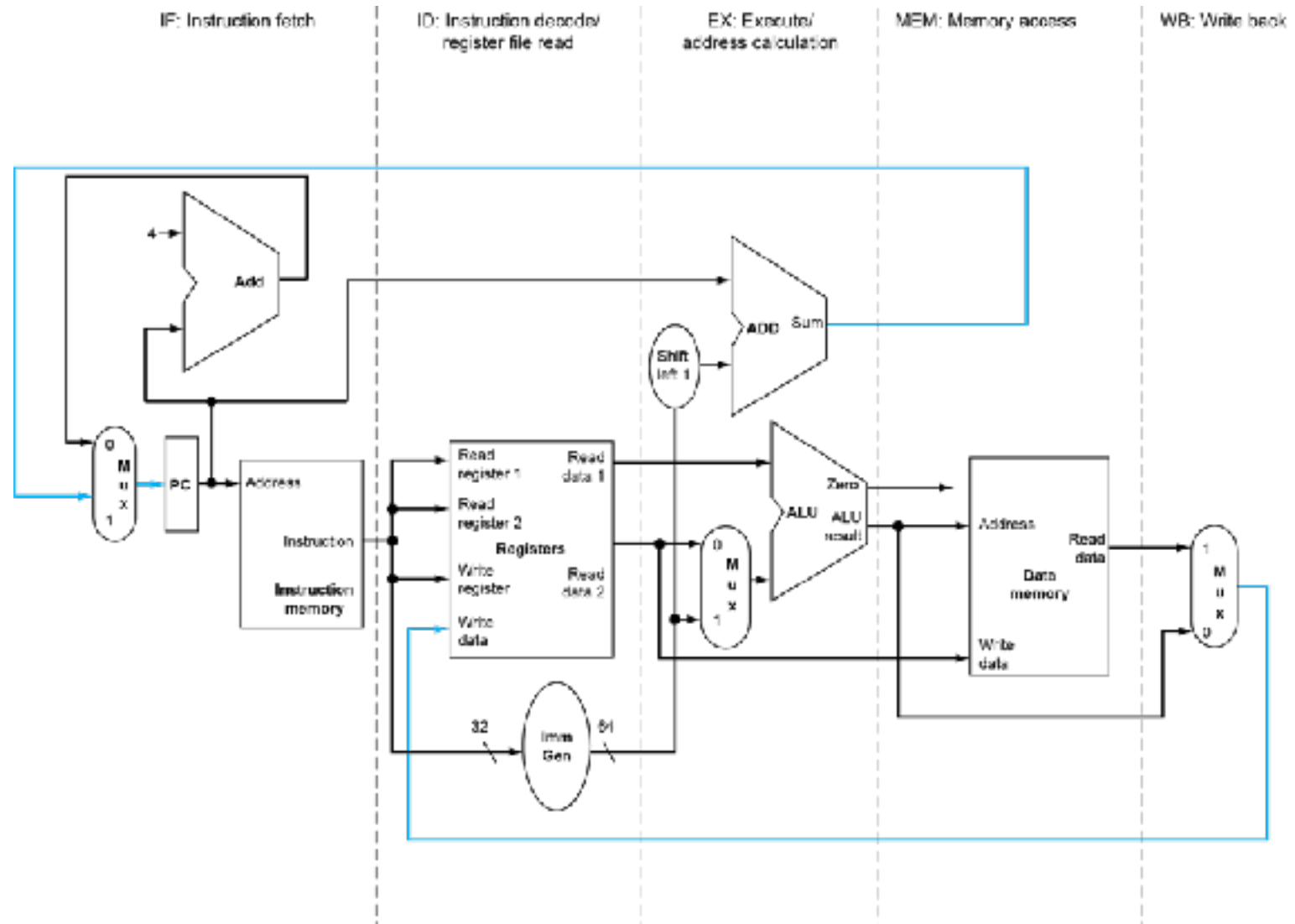
Universidade de Brasília

Departamento de Ciência da Computação

RISC-V Pipeline: Unidade Operativa e Controle



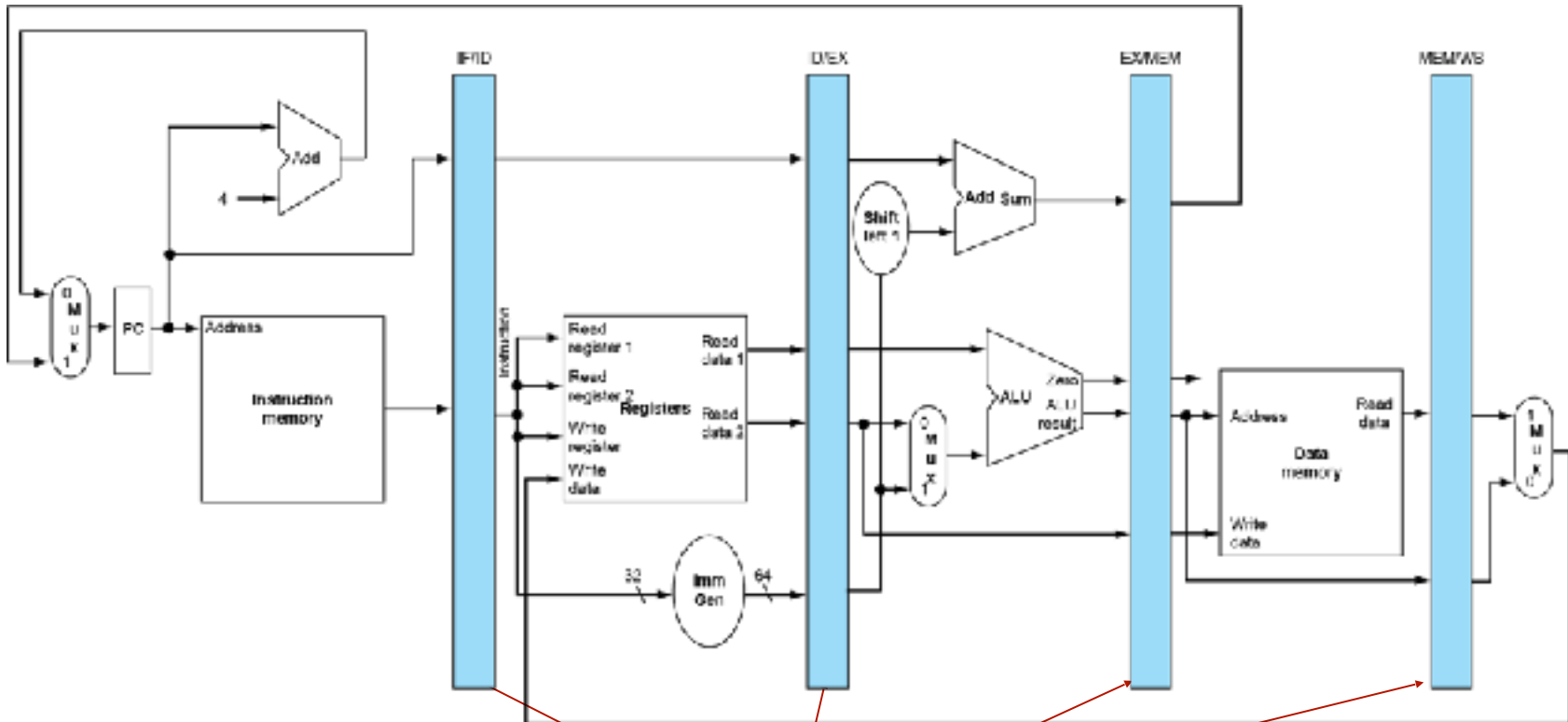
Caminho de dados uniclo





Caminho de dados com *pipeline*

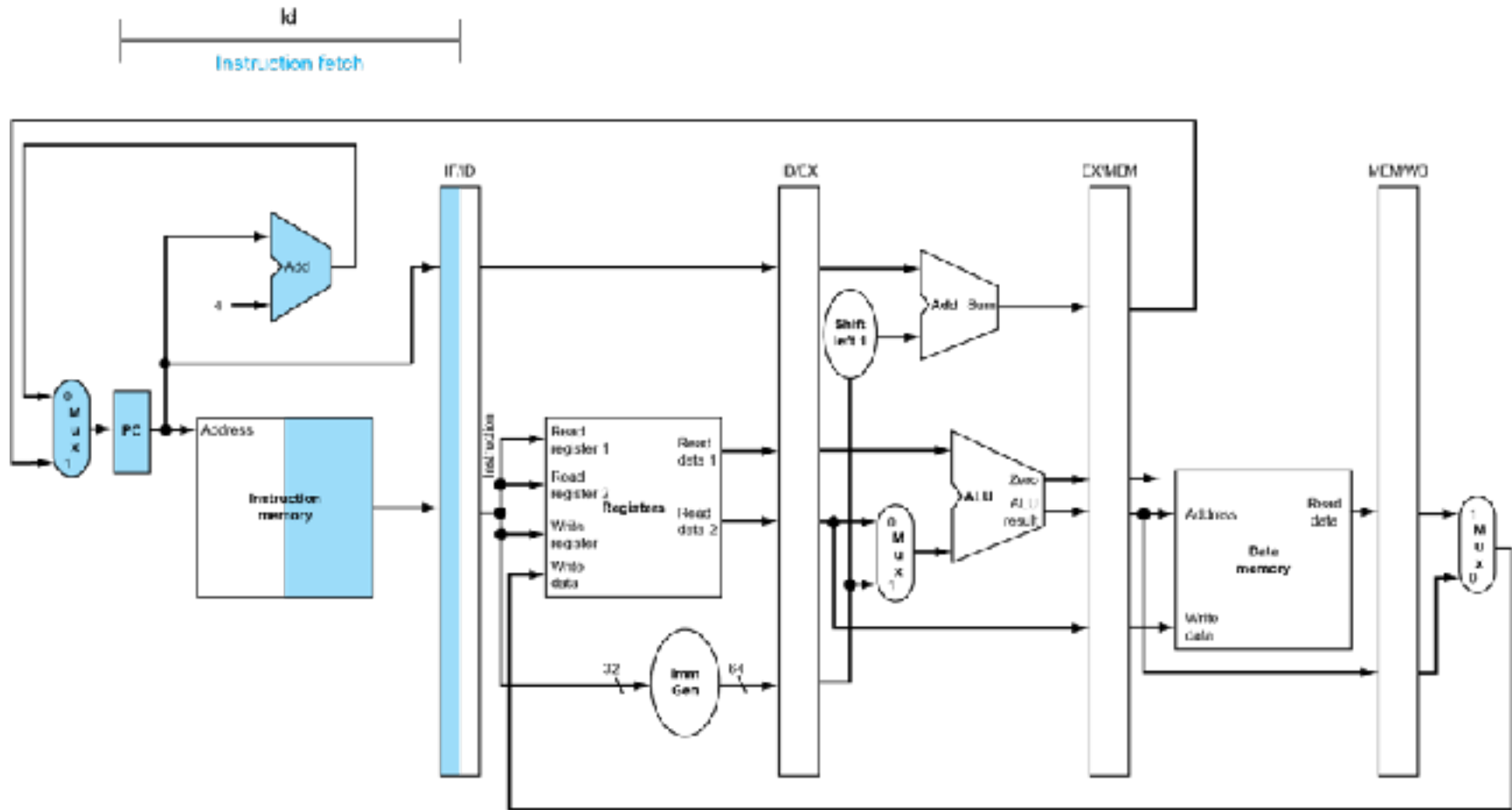
- Registradores armazenam valores dos estágios anteriores



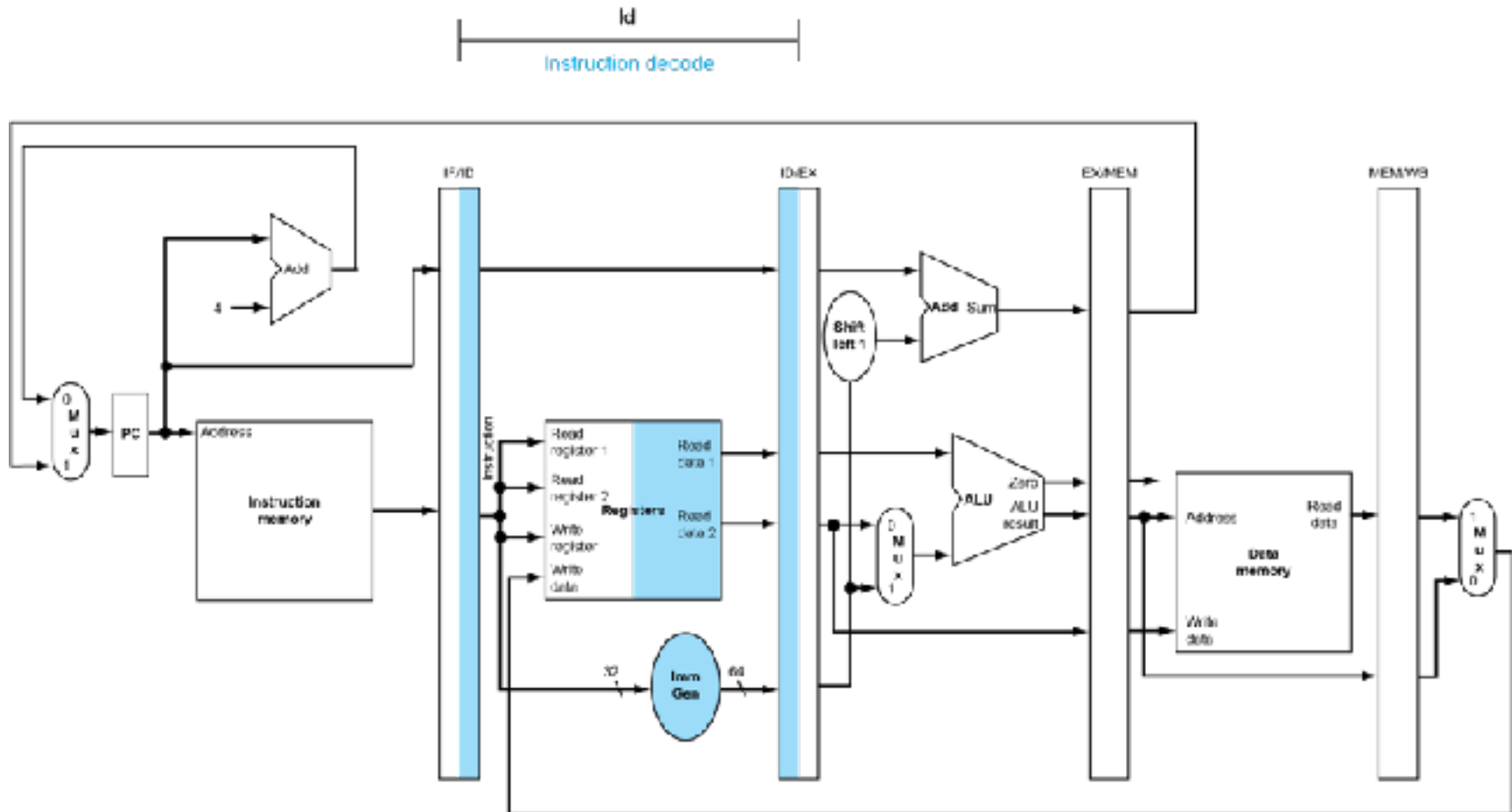
Registradores dos estágios do pipeline
Quais seus tamanhos?



Executando LW: busca

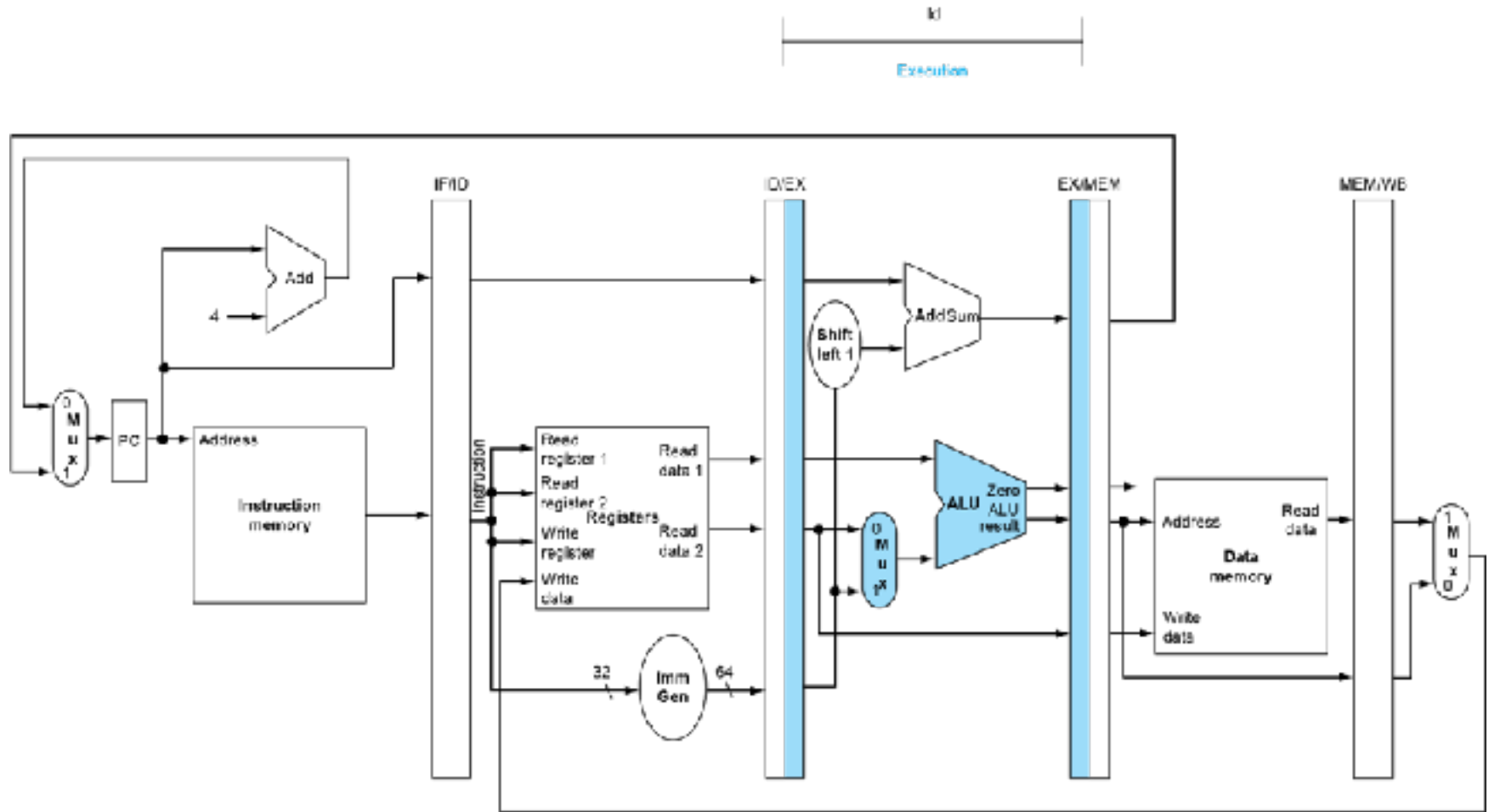


Executando LW: decodificação



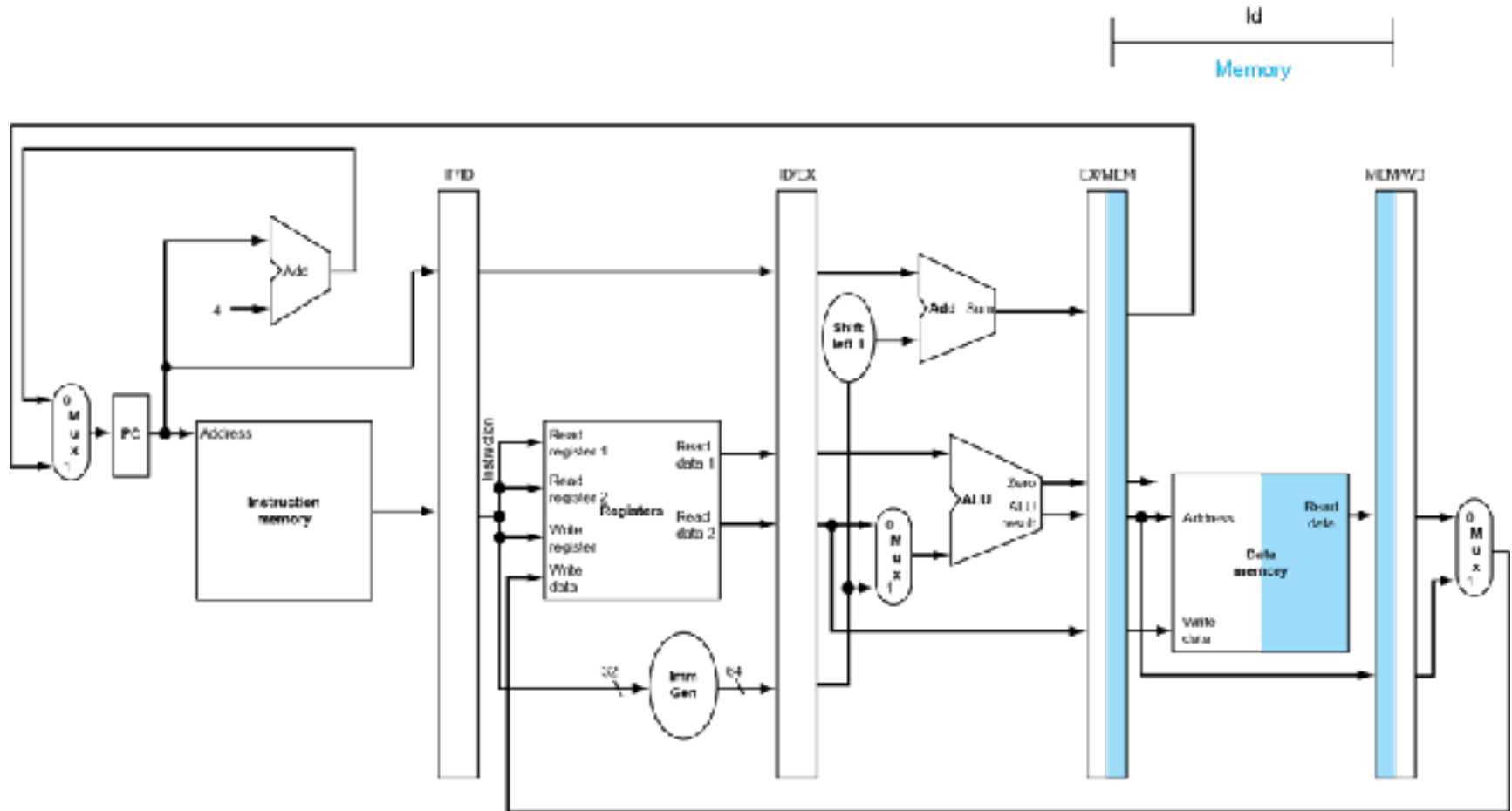


Executando LW: exec

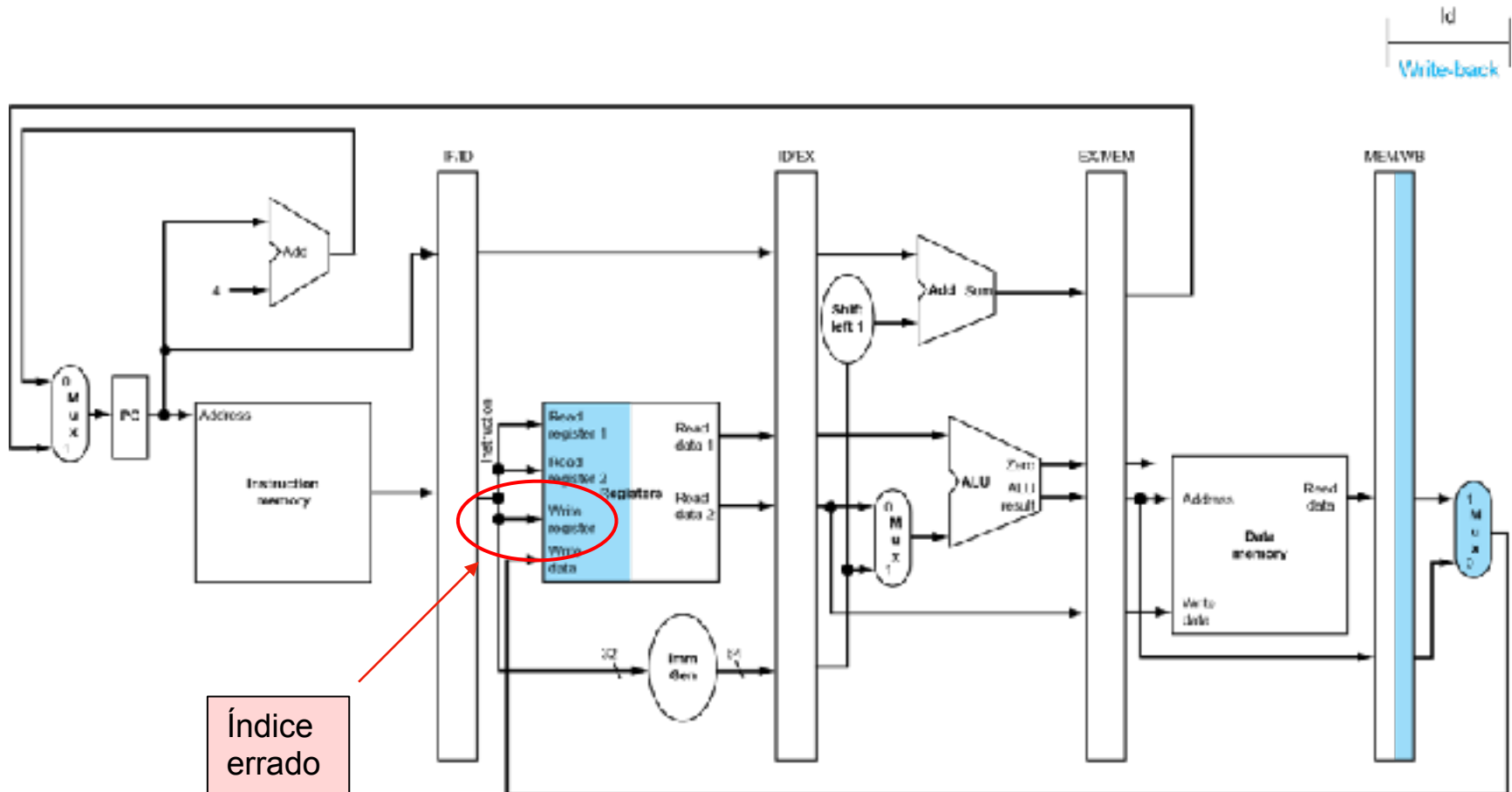




Executando LW: acesso memória

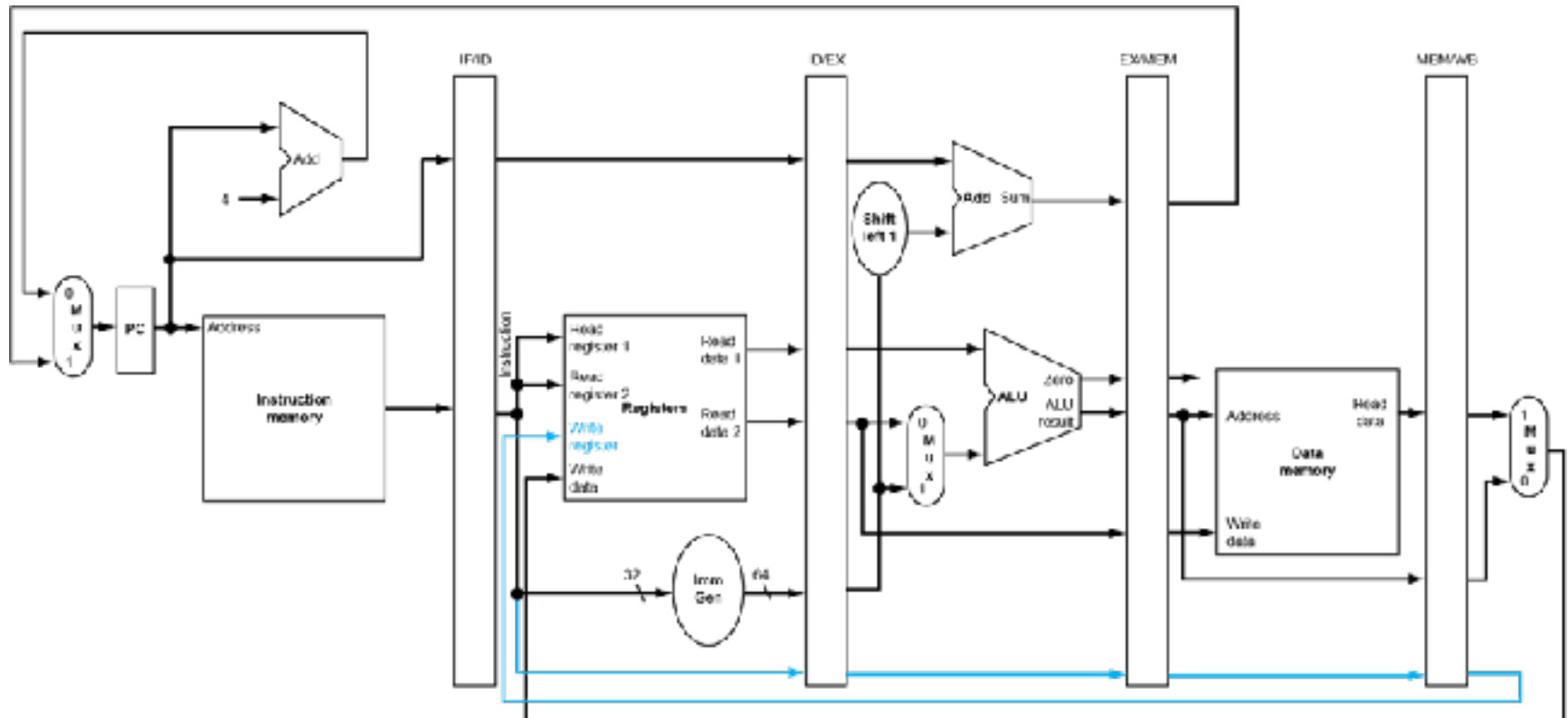


Executando LW: escreve registrador



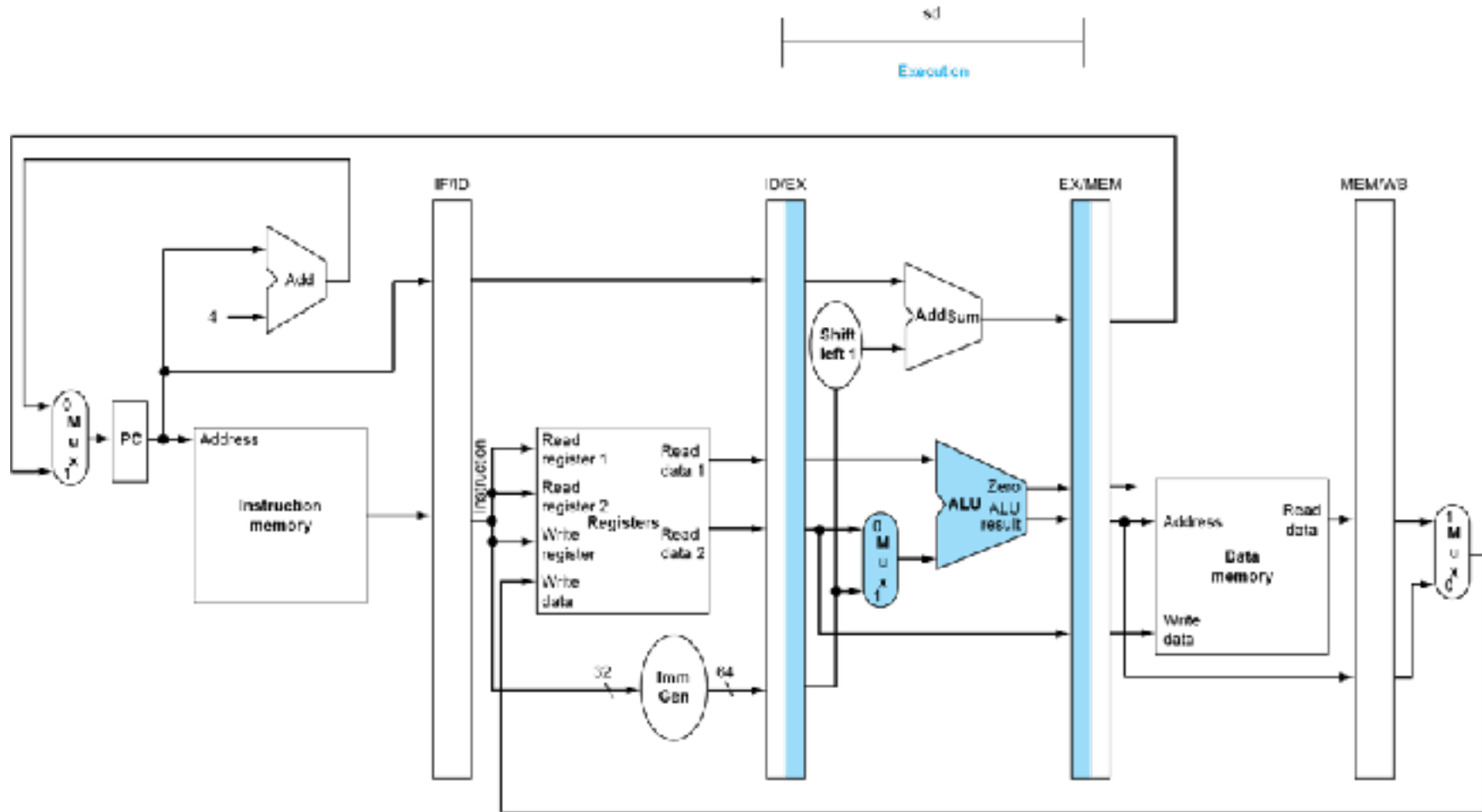


Corrigindo o erro:

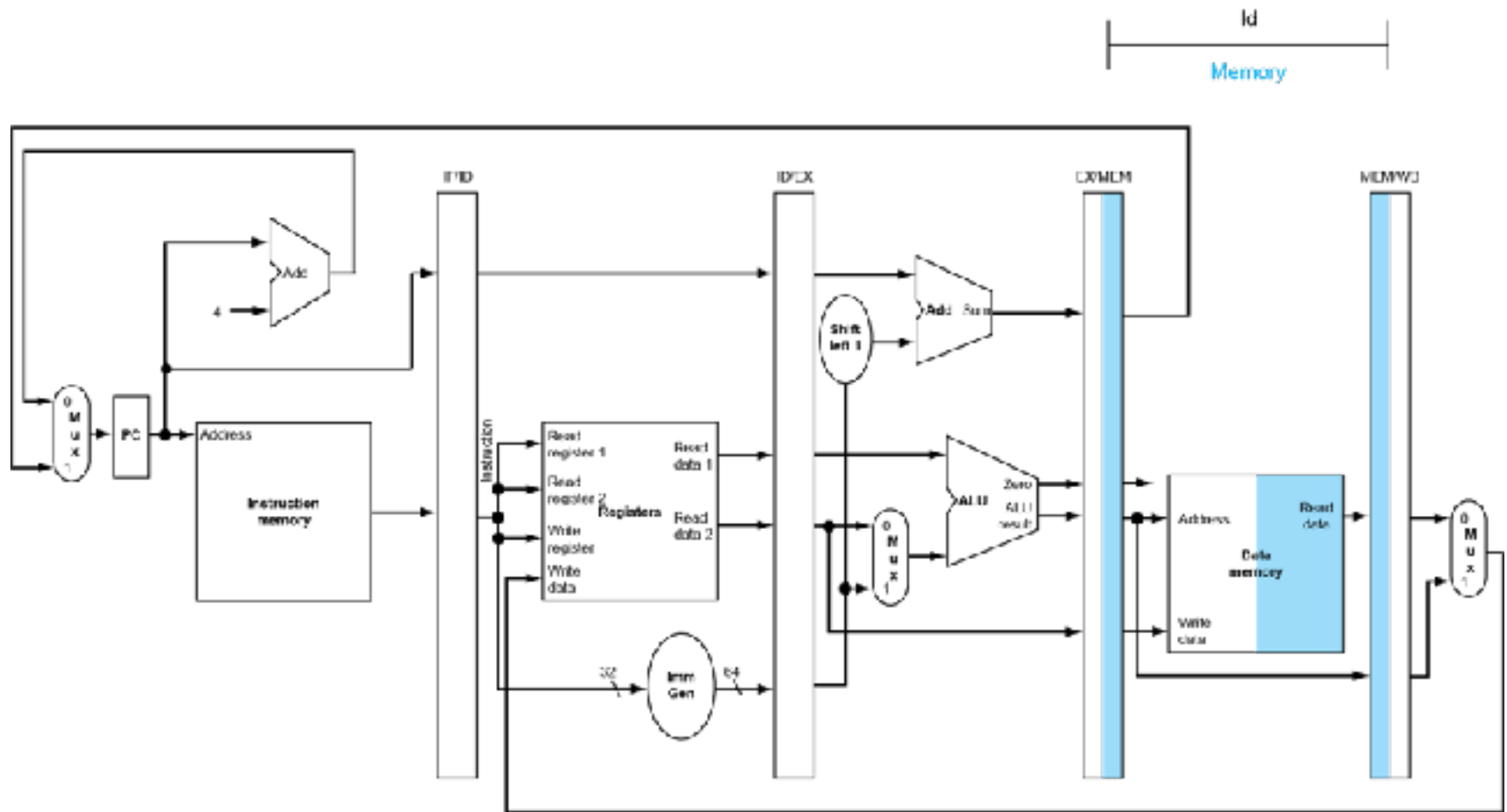




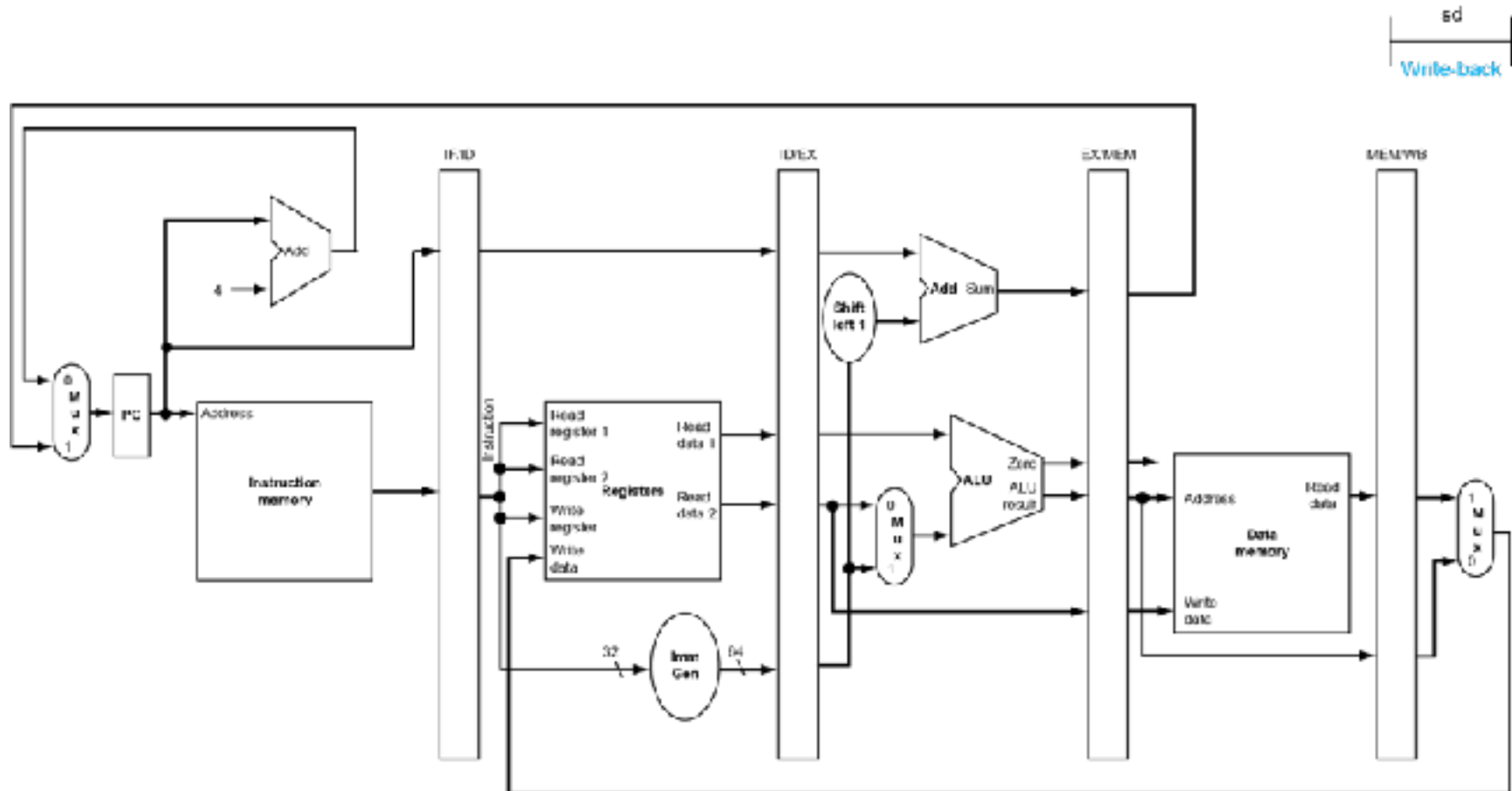
Execução do SW



Execução do SW: escreve MEM

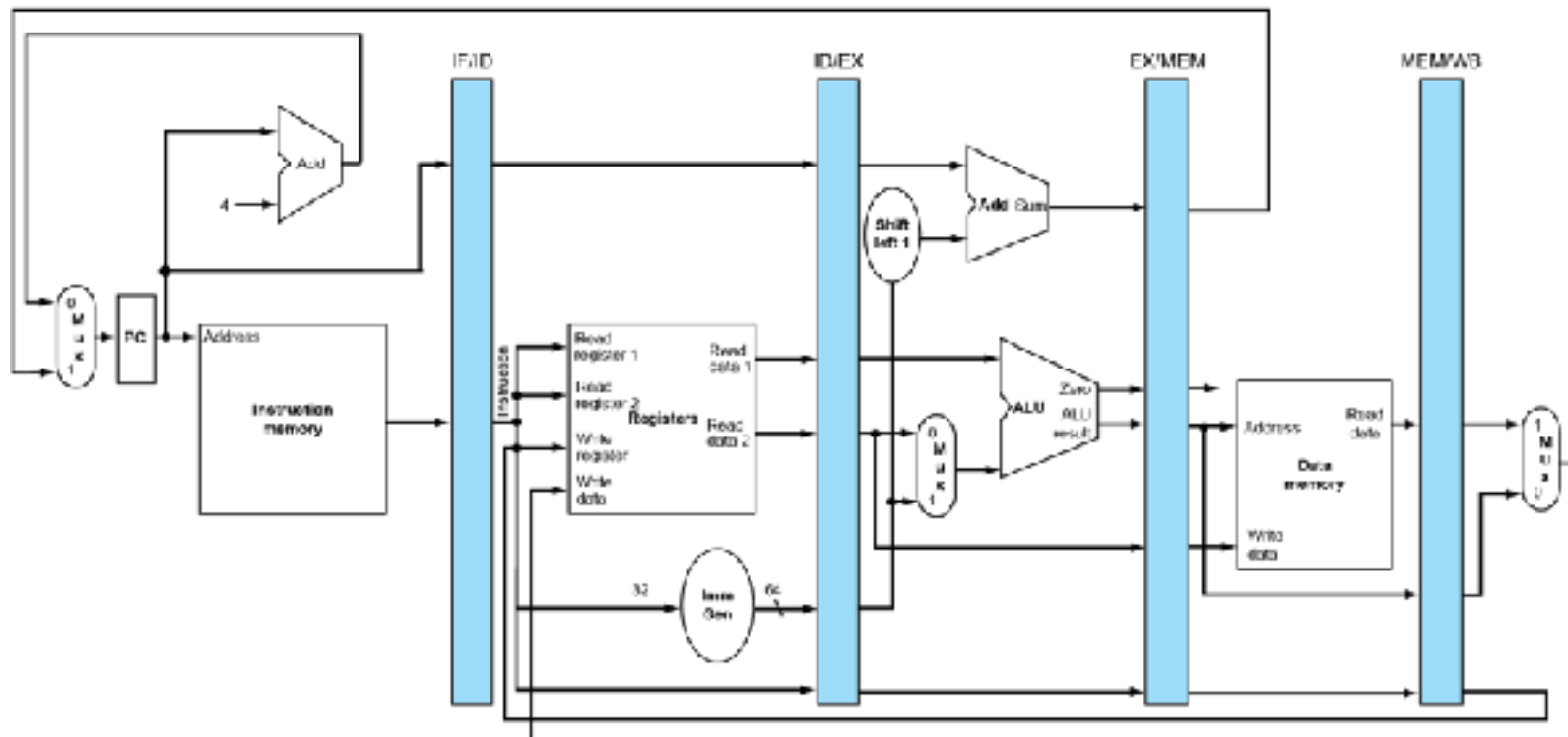


Execução do SW: escreve reg





Múltiplas Instruções no *Pipeline*





Controle do pipeline

- Temos 5 estágios. O que deve ser controlado em cada estágio?

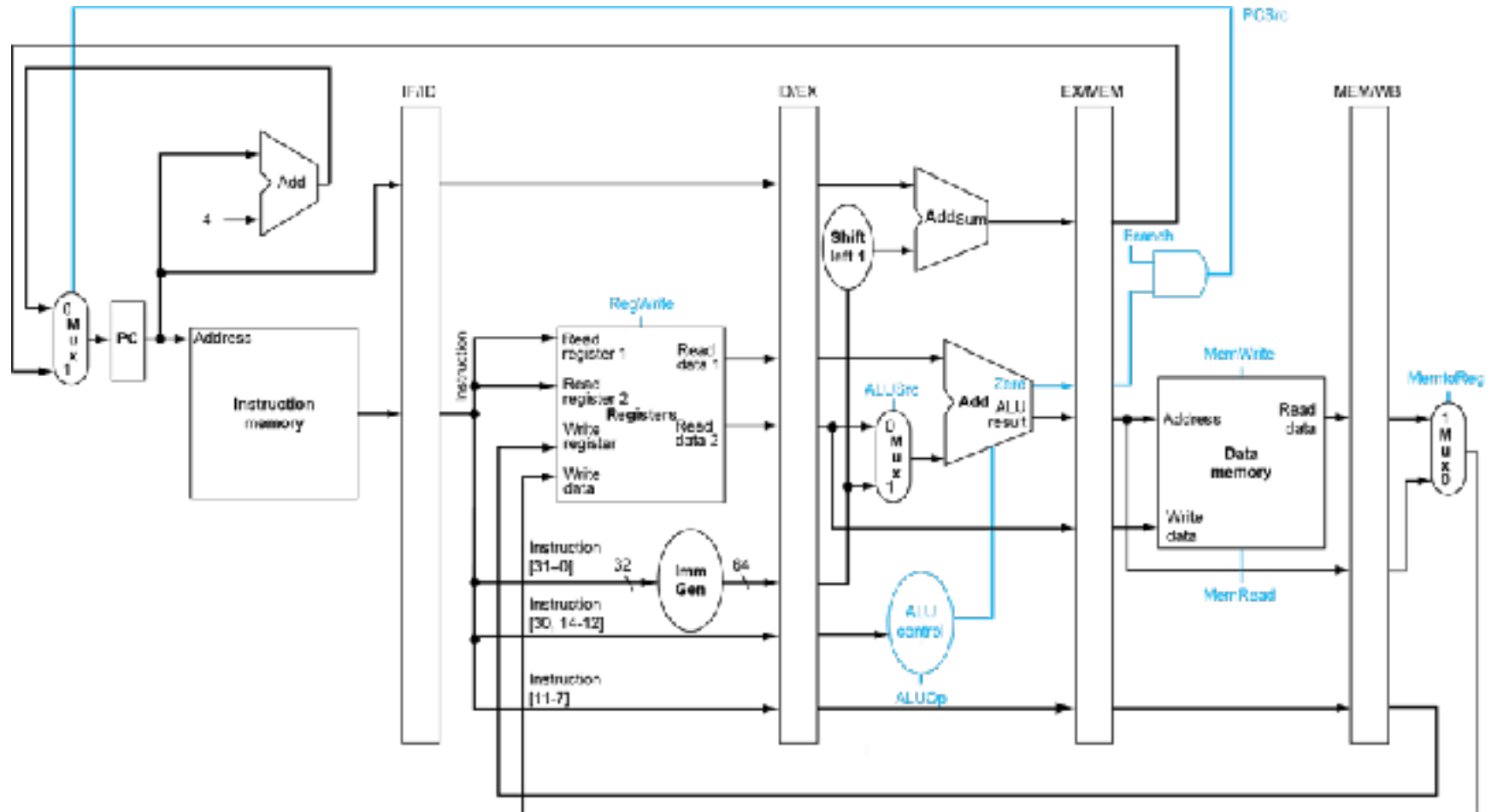
IF:	<i>Instruction Fetch e PC Increment</i>
ID:	<i>Instruction Decode / Register Fetch</i>
EX:	<i>Execution</i>
MEM:	<i>Memory Stage</i>
WB:	<i>Write Back</i>



Controle do pipeline

- Temos 5 estágios. O que precisa ser controlado em cada estágio?
 - Busca da instrução:
 - Nenhum sinal de controle
 - Decodificação da instrução / leitura do banco de regs.:
 - Nenhum sinal de controle
 - Execução / cálculo de endereço:
 - RegDest, OpALU, OrigALU
 - Acesso a memória:
 - Branch, LeMem, EscreveMem
 - Escrita do resultado
 - MemparaReg, EscreveReg

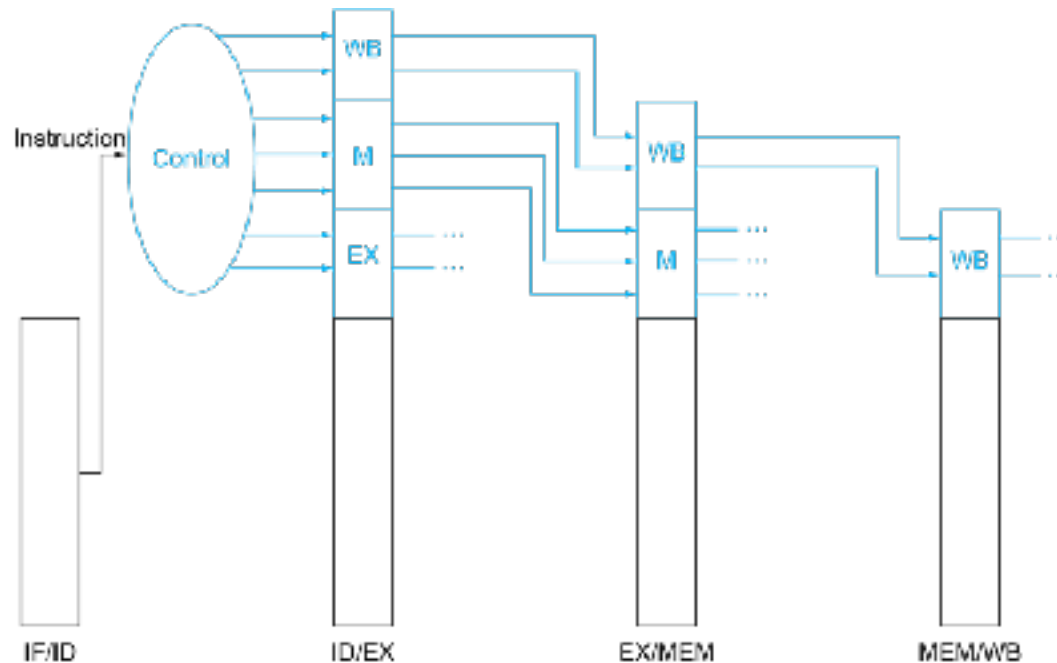
Controle do pipeline – Sinais de Controle



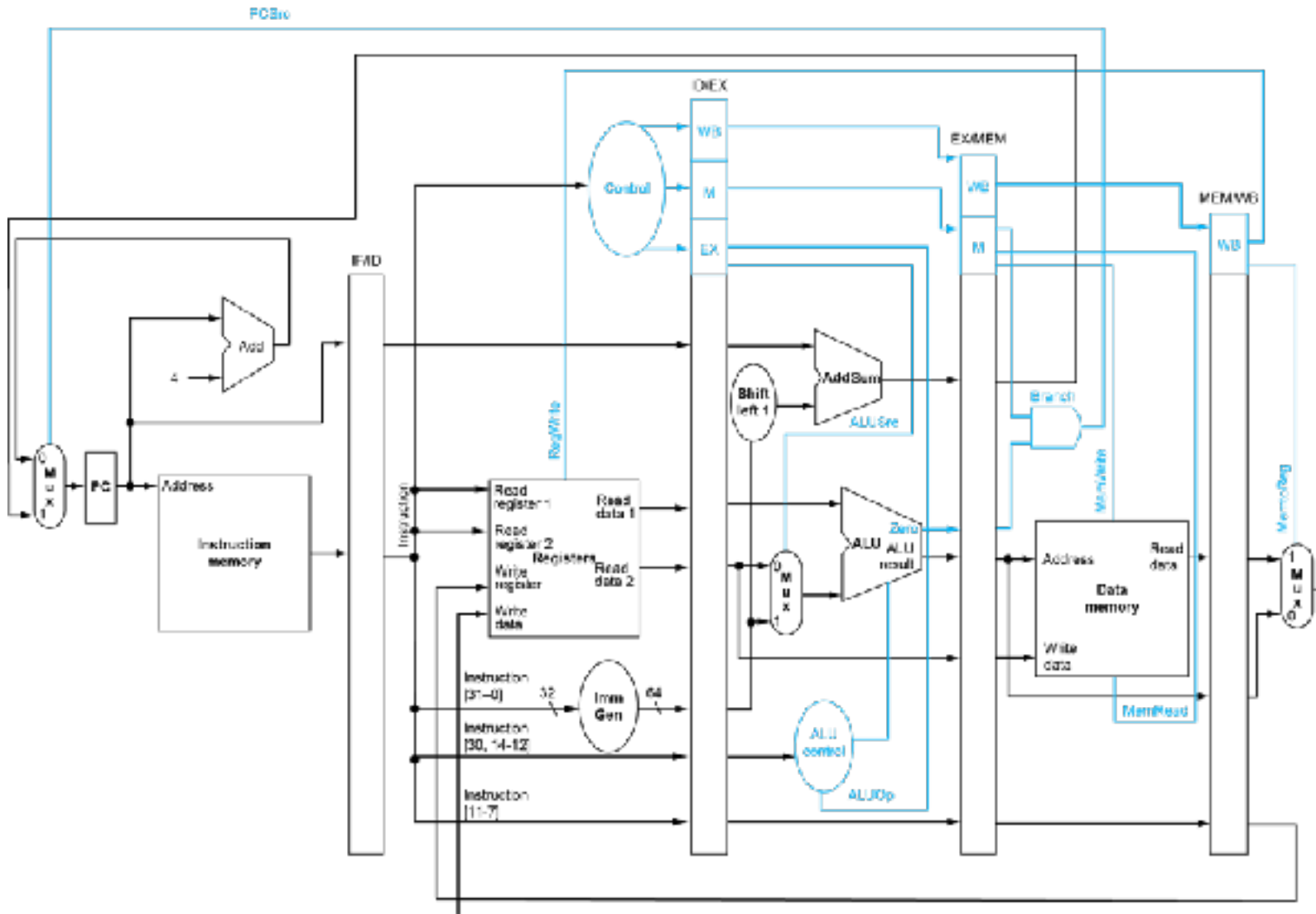
Controle do *Pipeline*

- Sinais de controle são gerados como no Uniciclo e transmitidos da mesma forma que os dados

	Estágio de EX		Estágio MEM			Estágio WB	
Instrução	ALUSrc	ALUOp	Branch	MemRead	MemWrite	RegWrite	Mem2Reg
Tipo-R	0	10	0	0	0	1	0
lw	1	00	0	1	0	1	1
sw	1	00	0	0	1	0	X
beq	0	01	1	0	0	0	X



Caminho de dados pipeline com controle





Adiantamento de Dados

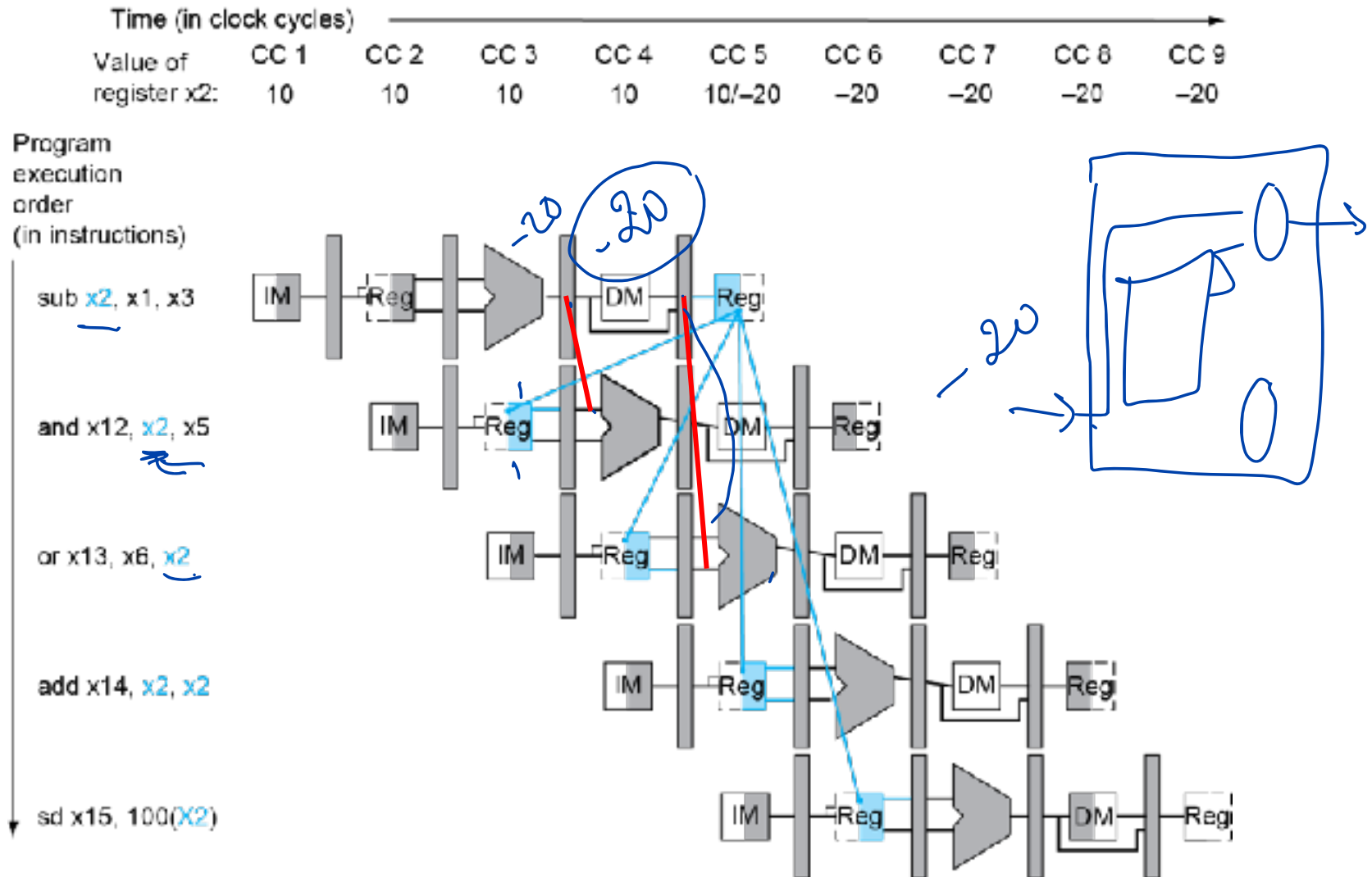
- Considerar a seguinte sequência de instruções:

```
sub  x2, x1, x3
and  x12, x2, x5
or   x13, x6, x2
add  x14, x2, x2
sd   x15, 100(x2)
```

- detectando e resolvendo os conflitos com adiantamento de dados

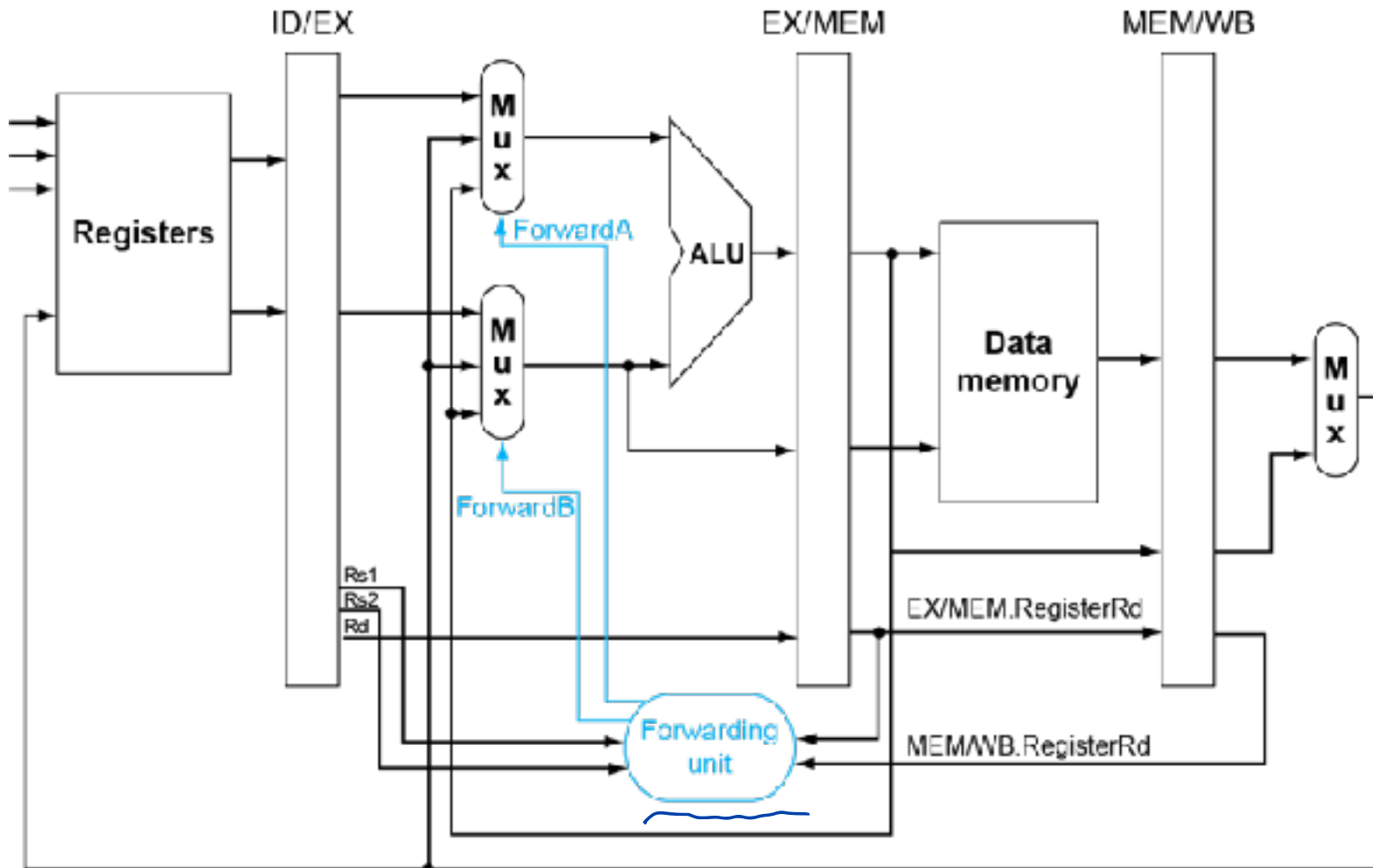
Conflitos

■ Conflito de Dados: Dependências e adiantamento

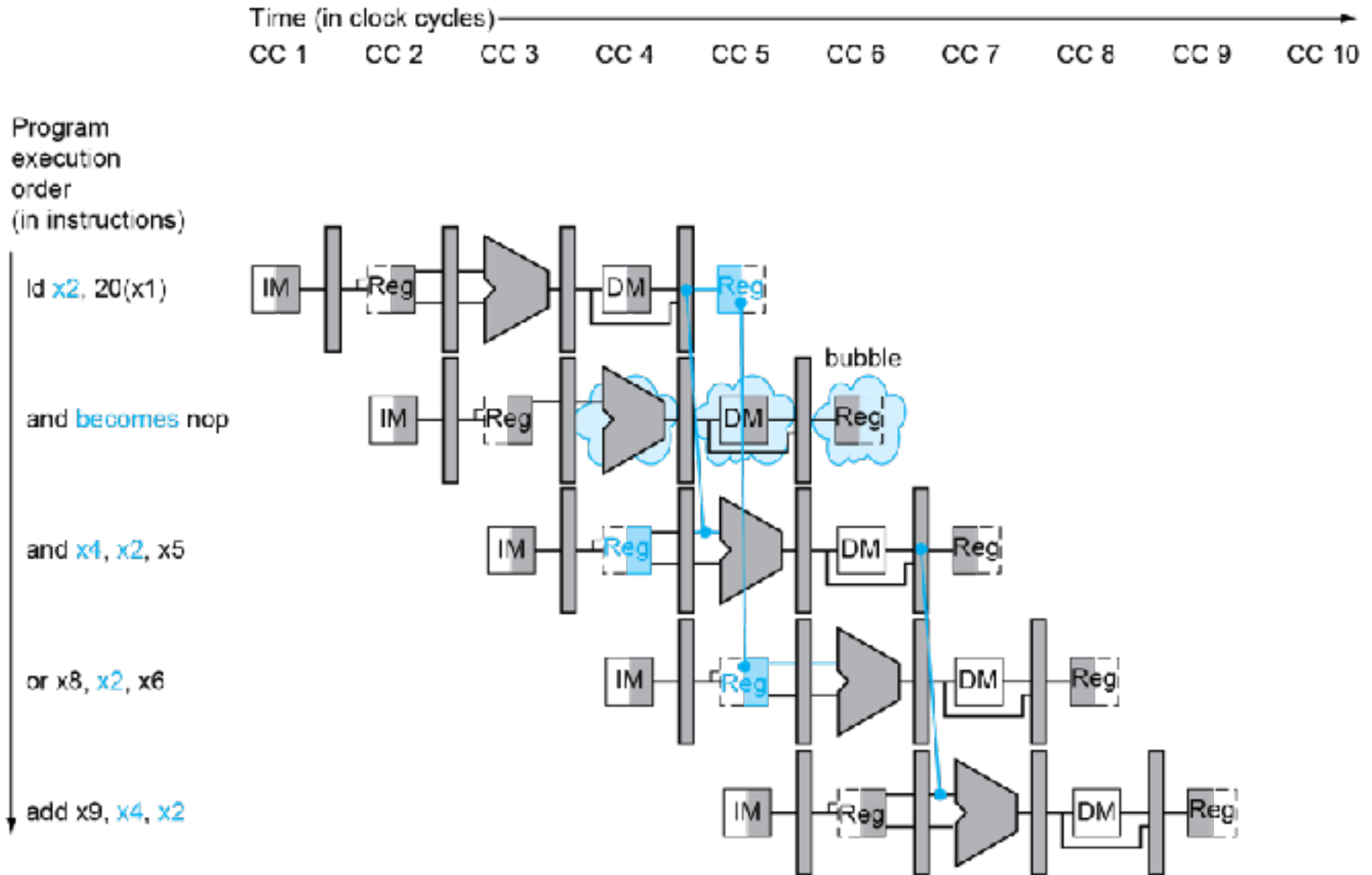


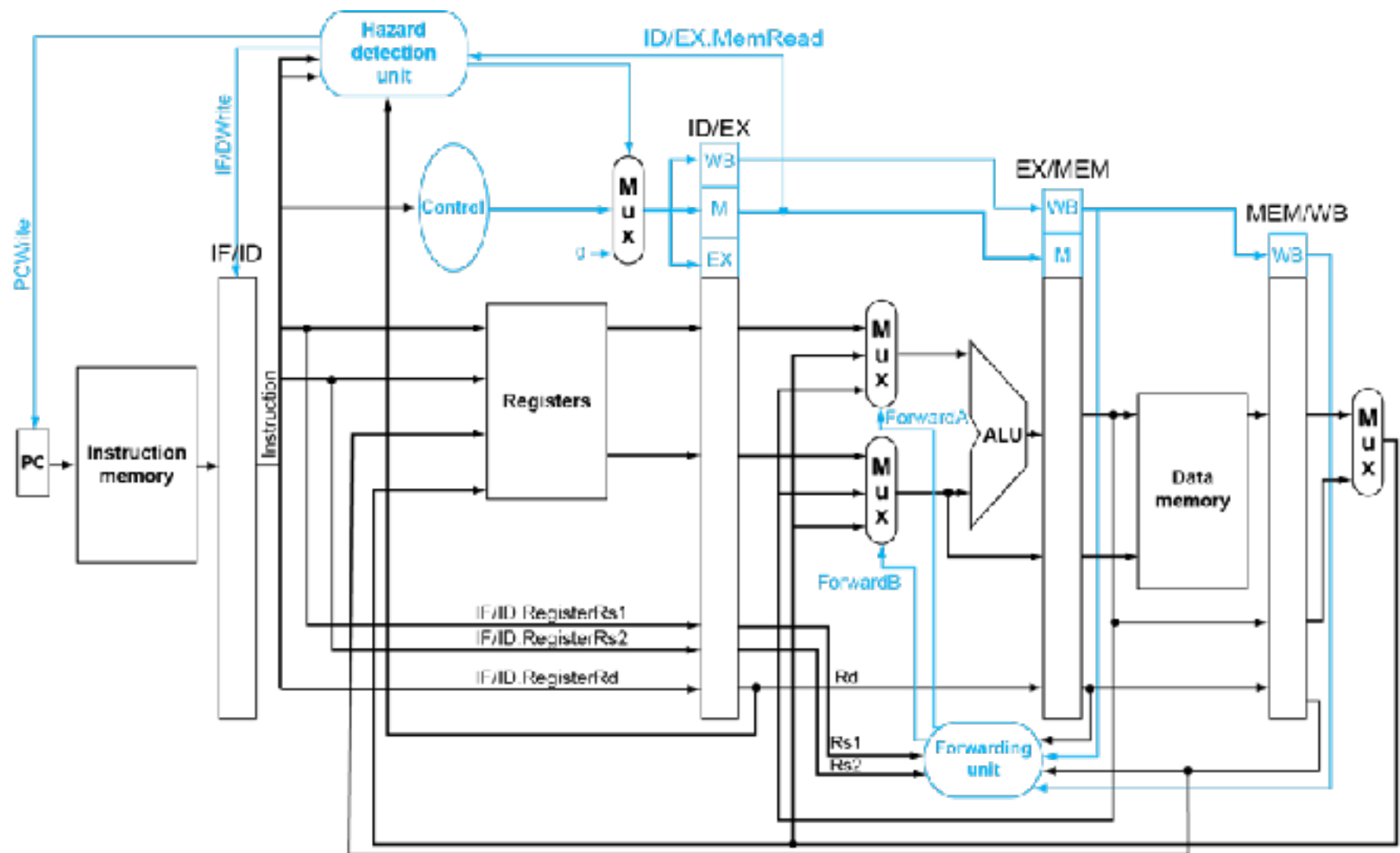


Conflito de dados: Adiantamento



Quando forward não é possível







Deteção de Hazards

- Comparar índices dos registradores a serem lidos e escritos, armazenados nos registradores do *pipeline*:
- Deve-se comparar igualmente se a instrução escreve em um registrador ($\text{EscreveReg} = 1$) ou lê da memória ($\text{LeMem} = 1$)
- Obs:
 - *Flush* descarta uma instrução do pipeline
 - *Stall* paraliza o pipeline



Detecção de Hazard de dados

- 1. Hazard EX
 - IF (EX/MEM.EscreveReg
and (EX/MEM.RegistradorRd $\neq$ 0)
and (EX/MEM.RegistradorRd = ID/EX.RegistradorRs1)) ForwardA = 10
 - IF (EX/MEM.EscreveReg
and (EX/MEM.RegistradorRd $\neq$ 0)
and (EX/MEM.RegistradorRd = ID/EX.RegistradorRs2)) ForwardB = 10

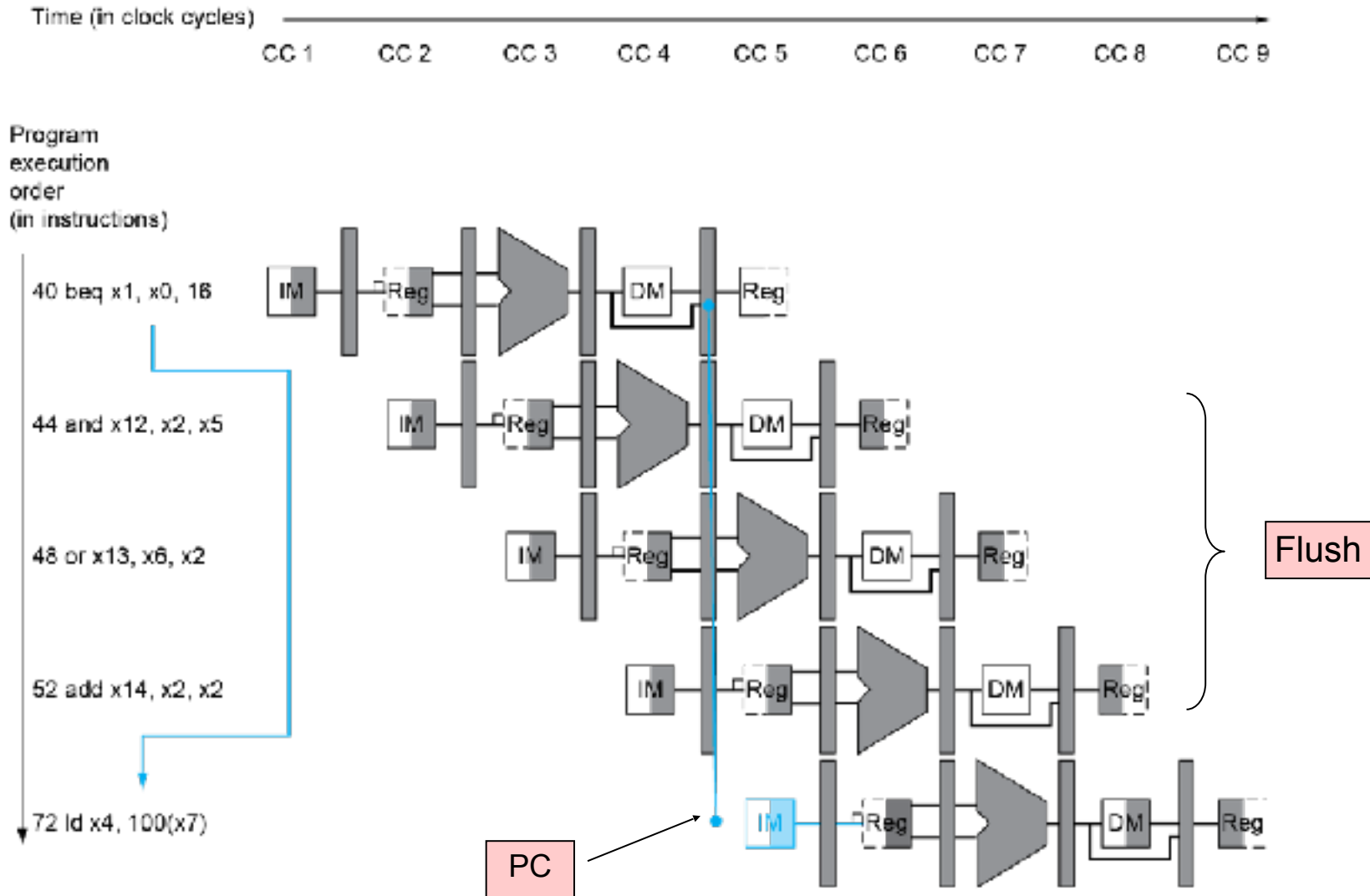
- 2. Hazard MEM
 - IF (MEM/WB.EscreveReg
and (MEM/WB.RegistradorRd $\neq$ 0)
and (MEM/WB.RegistradorRd = ID/EX.RegistradorRs1)) ForwardA = 01
 - IF (MEM/WB.EscreveReg
and (MEM/WB.RegistradorRd $\neq$ 0)
and (MEM/WB.RegistradorRd = ID/EX.RegistradorRs2)) ForwardB = 01

- 3. LW
 - IF (ID/EX.LeMem and ((ID/EX.RegistradorRd = IF/ID.RegistradorRs1)
or (ID/EX.RegistradorRd = IF/ID.RegistradorRs2))) Ocasiona stall no pipeline

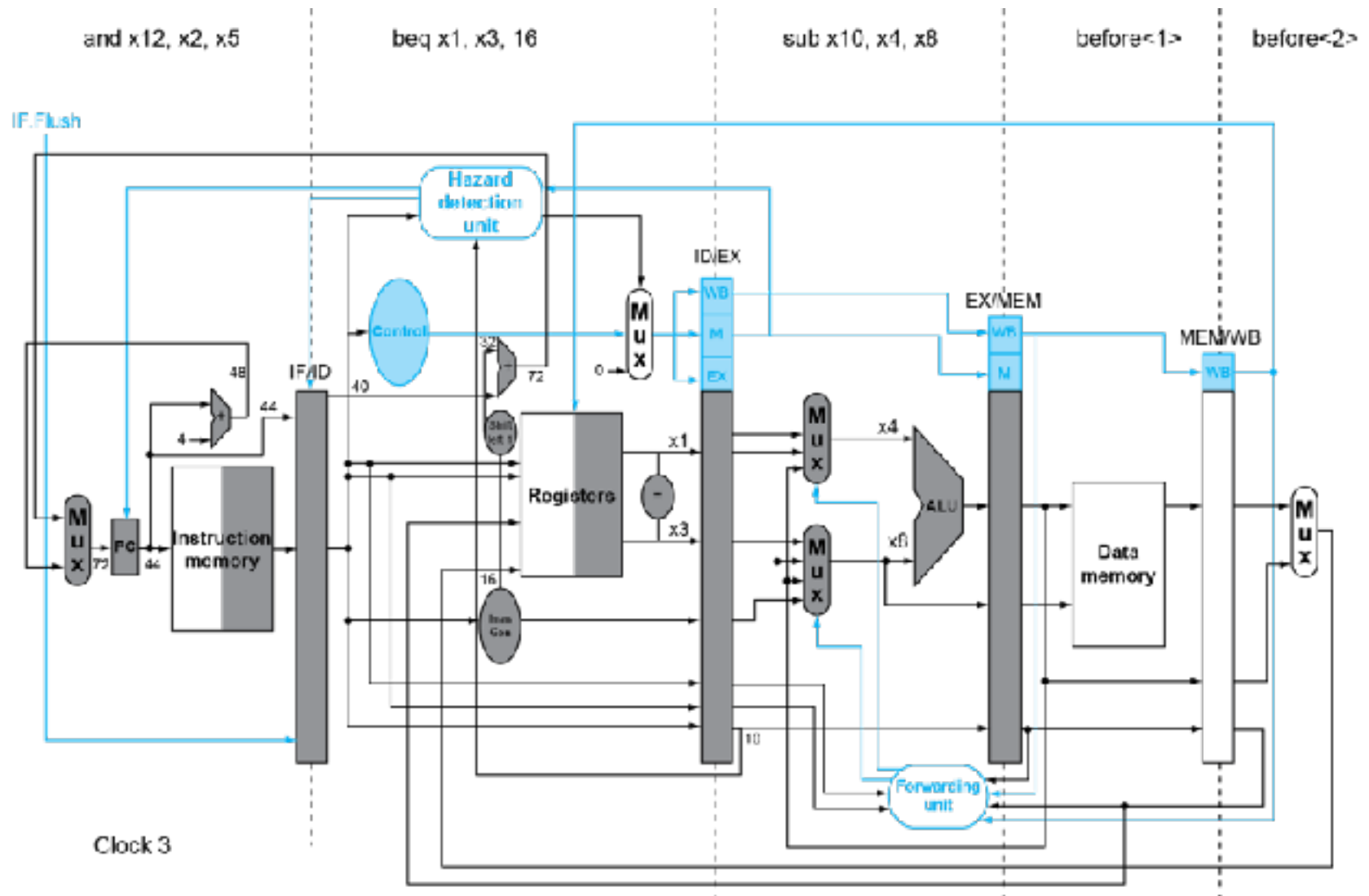


Hazard de Controle

■ Prevendo: Desvio não-tomado



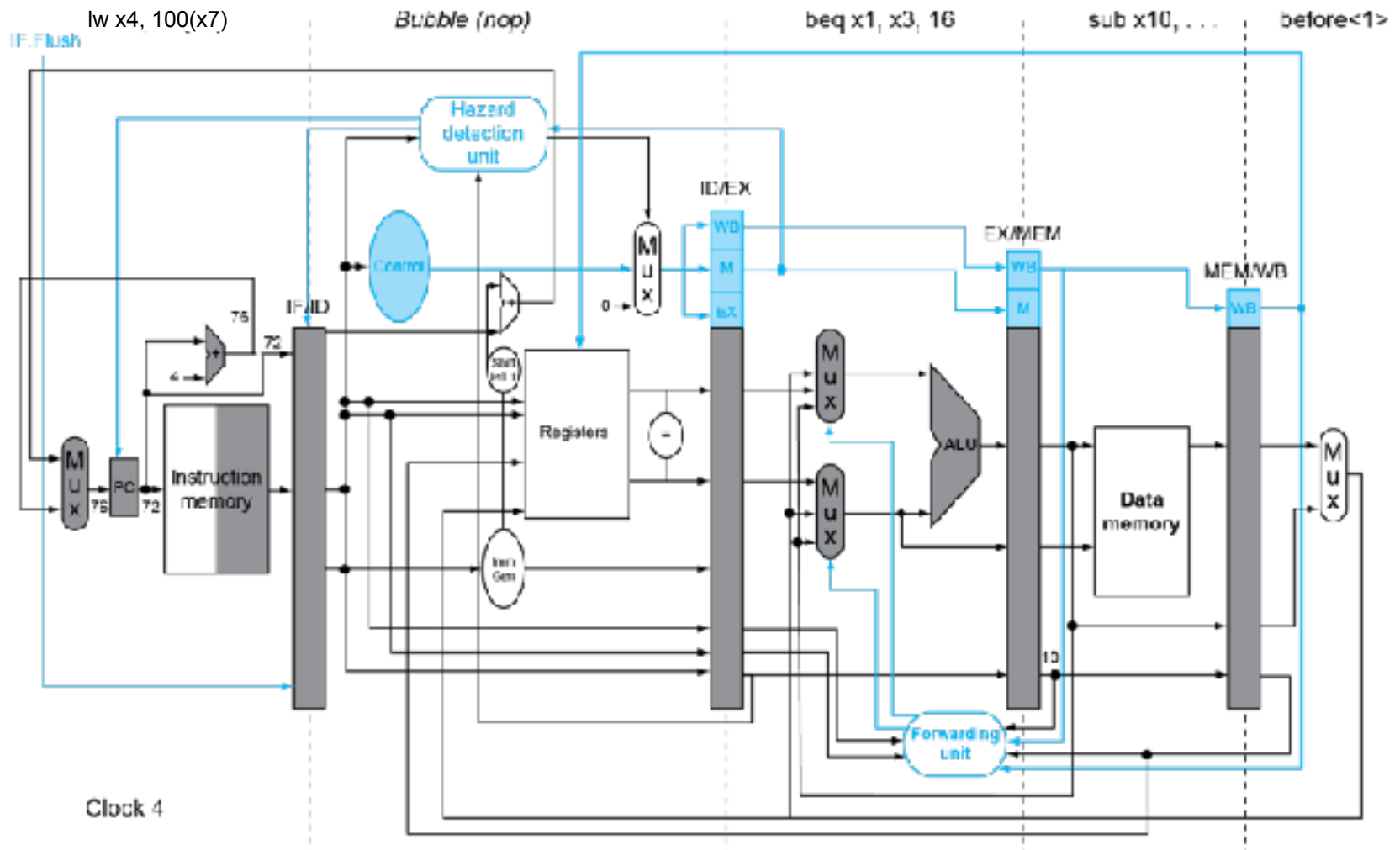
Hazard de Controle otimizado (1 bolha)





Hazard de Controle

- Hardware otimizado para 1 bolha





Comparação: Uniciclo x Multiciclo x Pipeline

- Suponha os seguintes tempos das unidades operativas:
 - Acesso a memória: 200ps
 - Operação da ULA: 100ps
 - Acesso ao banco de registradores: 50ps
- Dado o workload composto de:
 - 25% loads
 - 10% stores
 - 11% branches
 - 2% jumps
 - 52% operações com a ULA

Compare o desempenho, considerando para o pipeline

50% dos loads é seguido de uma operação que requer o argumento

25% dos desvios são previstos erradamente e que o atraso da previsão errada é 1 ciclo de clock

jumps usam 2 ciclos de clock

ignore outros hazards.



Solução

Ciclo único:

período de clock: $200+50+100+200+50 = 600\text{ps}$

$\text{CPI}=1$

Tempo de execução médio da instrução: $600 \times 1 = 600\text{ps}$

Multiciclo:

período de clock: 200ps

$\text{CPI} = .25 \times 5 + .1 \times 4 + .11 \times 3 + .02 \times 3 + 0.52 \times 4 = 4.12$

Tempo de execução médio da instrução: $200 \times 4.12 = 824\text{ps}$

Pipeline:

período de clock: 200ps

$\text{CPI} = .25 \times 1.5 + .1 \times 1 + .11 \times 1.25 + .02 \times 2 + 0.52 \times 1 = 1.17$

Tempo de execução médio da instrução: $200 \times 1.17 = 234\text{ps}$